



DESARROLLO BIG DATA CON PYTHON

TEMA 4. Spark SQL y
Procesamiento de Datos



Contenidos Tema 4

- / 1. Fundamentos de Spark SQL**
- / 2. Operaciones con DataFrames: selecciones, filtrado, agrupaciones y joins**
- / 3. Uso de funciones integradas y procesamiento**
- / 4. LAB: Limpieza y análisis exploratorio (EDA) con consultas SQL nativas**

¿Qué es Spark SQL?

- Un módulo central de Spark, especializado en el procesamiento de datos estructurados.
- El cerebro detrás del motor de Spark SQL tiene un nombre → **Catalyst Optimizer**: optimizador automático que reescribe tu código antes de su ejecución para maximizar el rendimiento.



- Gracias al optimizador **Catalyst**, el código se ejecuta con la máxima velocidad y eficiencia posibles, independientemente de cómo se hayan escrito aspectos como las consultas SQL.

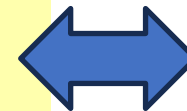
1. Fundamentos de Spark SQL

Spark SQL: Dualidad de Sintaxis

- **PySpark DataFrame API:** basado en ideas de Pandas y en orientación a objetos de Python. Más cómodo para desarrolladores y para modelar cadenas de transformaciones complejas.
- **SQL Nativo:** sintaxis clásica de consulta a bases de datos, más cómodo para analistas o para realizar consultas exploratorias rápidas y de menor complejidad.
- Dos enfoques **100% interoperables:** puedes empezar con la API de DataFrame para limpiar datos, y después agregarlos usando SQL si lo prefieres.

```
# --- Enfoque 1: DataFrame API (Python, similar a
Pandas) ---
df_resultado = df.filter(df.edad >= 18) \
    .groupBy("ciudad") \
    .count()

df_resultado.show()
```



```
# --- Enfoque 2: SQL Nativo ---
df_resultado = spark.sql("""
    SELECT ciudad, COUNT(*) as count
    FROM usuarios
    WHERE edad >= 18
    GROUP BY ciudad
""")

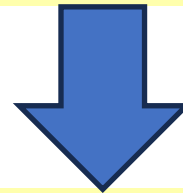
df_resultado.show()
```

```
+-----+-----+
| ciudad|count|
+-----+-----+
| London| 2|
|New York| 3|
+-----+-----+
```

Spark SQL: Vistas Temporales

- Spark SQL no puede leer un DataFrame de Python directamente usando código SQL.
- **Solución:** creación de una **vista temporal**, registrando un DataFrame en el catálogo de metadatos de Spark → esto permite que los datos sean "visibles" a un comando SQL como si de una tabla se tratara.

```
# Registramos el DataFrame como una vista temporal para poder utilizar Spark SQL
df.createOrReplaceTempView("usuarios")
```



```
# --- Enfoque 2: SQL Nativo ---
df_resultado = spark.sql("""
    SELECT ciudad, COUNT(*) as count
    FROM usuarios
    WHERE edad >= 18
    GROUP BY ciudad
    """)
```

2. Operaciones con DataFrames en PySpark

SELECCIÓN Y DERIVACIÓN DE DATOS

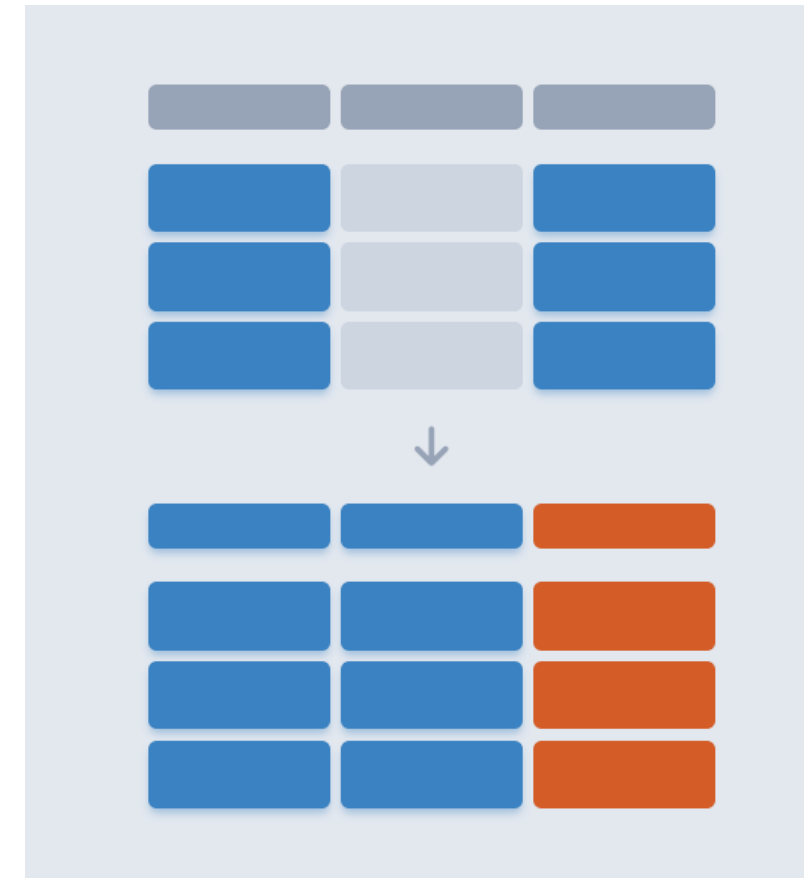
Extraer columnas específicas y crear nuevas variables a partir de los datos existentes.

Seleccionar dos columnas por nombre y crear una nueva columna resultado de aplicar IVA del 21% a los valores de la columna "ventas":

```
from pyspark.sql.functions import col, round

# 1. DataFrame API
df.select(
    col("id_usuario"), col("ventas")
).withColumn("con_iva", round(col("ventas") * 1.21, 2)).show()
```

```
# 2. Spark SQL
spark.sql("""
    SELECT id_usuario, ventas,
    (ventas * 1.21) AS con_iva
    FROM tabla_ventas
    """).show()
```



2. Operaciones con DataFrames en PySpark

FILTRADO DE DATOS

Extraer únicamente las filas cuyos valores cumplan alguna condición lógica especificada.

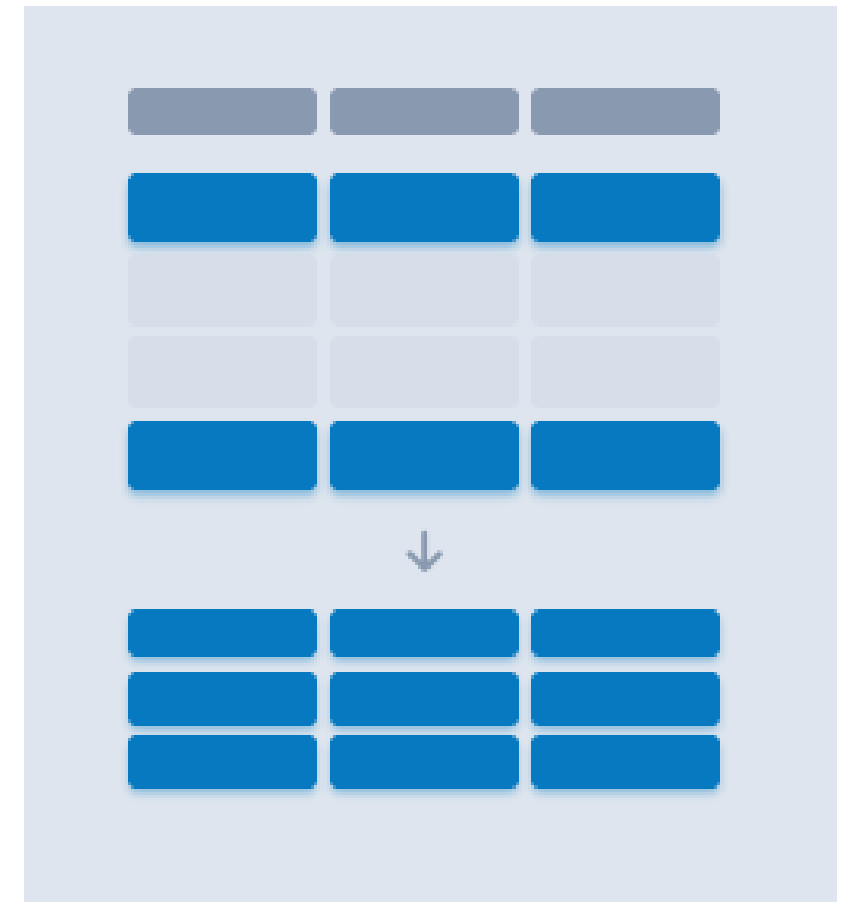
Seleccionar los datos de vendedores cuyo número de ventas es superior a 120

```
# 1. DataFrame API: Usando col() para referenciar la columna
from pyspark.sql.functions import col
df_filtrado = df.filter(col("ventas") > 120)

df_filtrado.show()
```

```
# 2. Spark SQL Nativo: la cláusula WHERE clásica de SQL
df_filtrado_sql = spark.sql("""
SELECT *
FROM tabla_ventas
WHERE ventas > 120
""")

df_filtrado.show(3)
```



2. Operaciones con DataFrames en PySpark

AGRUPACIÓN DE DATOS

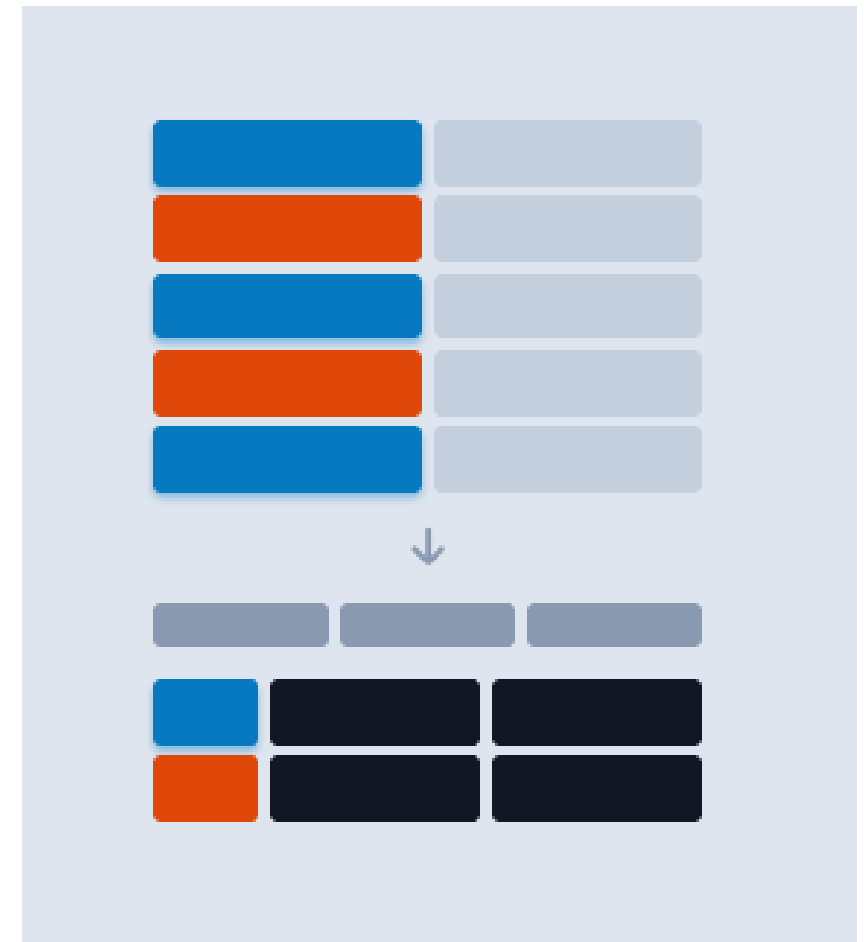
Crear grupos de datos (filas) bajo una categoría común, y aplicar sobre cada grupo una función matemática o de agregación (suma, recuento, media, etc.).

```
# 1. DataFrame API: Agrupar por ciudad y sumar la
columna ventas
df_agrupado = df.groupBy("ciudad").sum("ventas")

df_agrupado.show()
```

```
# 2. Spark SQL Nativo: Usar GROUP BY y la función de
agregación SUM()
df_agrupado_sql = spark.sql("""
SELECT ciudad, SUM(ventas) as total_ventas
FROM tabla_ventas
GROUP BY ciudad
""")

df_agrupado.show()
```



2. Operaciones con DataFrames en PySpark

CRUCE DE DATOS: JOIN

Enriquecer un DataFrame combinando datos de múltiples DataFrames mediante correspondencia entre claves de enlace (similar a los JOIN entre tablas SQL).

```
# 1. DataFrame API: cruzamos
usando la columna común "ciudad"
df_join = df.join(df_zonas,
on="ciudad", how="inner")
```

```
df_join.show()
```

```
# 2. Spark SQL Nativo: Usando INNER JOIN
estándar
```

```
df_join_sql = spark.sql("""
    SELECT v.id_usuario, v.ventas,
    v.ciudad, z.zona
    FROM tabla_ventas v
    INNER JOIN tabla_zonas z
    ON v.ciudad = z.ciudad
""")
```

```
df_join.show()
```

Datos de muestra de df:

id_usuario	ventas	ciudad
1	100	Madrid
2	150	Barcelona
3	200	Valencia
4	50	Sevilla
5	120	Madrid

only showing top 5 rows

ciudad	zona
Madrid	Centro
Barcelona	Este

Modos de cruce:

inner, left,
right, outer

